FinnVocab Project Report
Finnish Language Learning Web Application
Prepared by: Omole Peter Mayowa
Date: 20 April 2026

1. Project Description and Problem Definition

FinnVocab is a web-based language learning application designed to support non-native speakers living in Finland, especially new immigrants and international students, in building practical Finnish vocabulary for real daily situations. The project addresses a common learning problem: many beginner language resources are either too academic, too generic, or not focused on everyday communication needs such as work, transport, shopping, emergency situations, and social interaction.

The main problem identified was that learners often encounter Finnish words in context (for example, at work or in shops) but do not have a lightweight, personal tool to quickly capture, revise, and test these words. Traditional methods (paper notes, scattered phone notes, or static word lists) do not provide enough interaction, repetition, or feedback. FinnVocab was created to solve this gap by combining flashcards, quizzes, phrasebook guidance, grammar support, and custom vocabulary management in one simple interface that runs directly in a browser.

The project goals were:

- Provide a clean and responsive learning interface accessible on desktop and mobile devices.
- Support active recall through flashcards and quiz modes.
- Allow users to add and manage personal vocabulary (not only preloaded words).
- Include practical phrase and grammar support for contextual learning.
- Use modern web architecture principles: separation of concerns, asynchronous data access, and deployable frontend/backend components.

The final application demonstrates core learning workflows: viewing categorized vocabulary, reviewing words with examples and phonetics, taking quizzes, tracking progress points, and storing personal additions locally. It is publicly deployable and usable without local installation, meeting the course requirement for real-world accessibility.

2. Architecture Overview

FinnVocab follows a layered architecture with clear separation between user interface, application logic, data handling, and backend API services. The deployed version supports both API-driven content loading and fallback to embedded local data if the API is unavailable. This improves resilience for static-hosting scenarios while still demonstrating proper asynchronous API integration.

2.1 Architecture Diagram

```
+----------------------------- Client (Browser) ------------------------------+
| index.html + css/style.css                                                  |
| - Semantic UI structure                                                     |
| - Responsive styling                                                        |
|                                                                             |
| js/app.js                                                                   |
| - Navigation, screens, events, state, rendering                             |
| - Quiz/flashcard logic, localStorage progress                               |
|                                                                             |
| js/data-service.js + js/config.js                                           |
| - fetch('/api/words'), fetch('/api/phrases'), fetch('/api/grammar')         |
| - Error handling + fallback mode                                            |
+-----------------------------------|-----------------------------------------+
                                    | HTTPS (JSON via fetch)
                                    v
+---------------------------- Serverless/API Layer ----------------------------+
| Vercel Functions (api/*.js)                                                 |
| /api/health, /api/words, /api/phrases, /api/grammar                         |
| - Read JSON files from api-data/                                            |
| - Return structured JSON responses                                          |
+-----------------------------------|-----------------------------------------+
                                    v
+------------------------------- Data Storage ---------------------------------+
| api-data/words.json                                                         |
| api-data/phrases.json                                                       |
| api-data/grammar.json                                                       |
+-----------------------------------------------------------------------------+
```

2.2 Flow Summary

1) User opens the web app URL.
2) UI initializes and invokes the data service.
3) Data service requests JSON from API endpoints asynchronously using *fetch*.
4) If API is reachable, remote JSON content populates starter words, phrases, and grammar sections.
5) If API fails, app falls back to built-in local data so core features remain usable.
6) User interactions (quiz score, streak, custom words) are managed in client state and persisted to localStorage.

3. Technology Choices and Justification

Frontend:HTML5, CSS3, Vanilla JavaScript

- HTML5 was selected to provide semantic structure and accessibility-friendly markup.
- CSS3 was used for responsive layout, theming (including dark mode), component styling, and transitions without introducing framework overhead.
- Vanilla JavaScript was chosen deliberately to show direct competency in DOM manipulation, event handling, state updates, and rendering logic without relying on React/Vue abstractions.

Data Format and Communication:JSON + Fetch API

- JSON is lightweight, human-readable, and well-suited for exchanging vocabulary and phrase data.
- Fetch API demonstrates asynchronous client-server communication required by the technical specification.
- Error handling in data-service.js provides robustness and educational value by showing production-aware behavior.

Backend: Node.js / Express and Vercel Serverless API

- Node.js + Express was implemented as the minimum required backend pattern during development.
- For deployment compatibility, Vercel serverless API endpoints were added. This avoids persistent server hosting complexity while still fulfilling API requirements.
- API routes are small, maintainable, and centered around data retrieval from JSON files.

Deployment: Vercel

- Vercel was selected because it supports both static frontend hosting and serverless functions in one URL, making the project easier to share with teachers and evaluators.
- Continuous deployment integration with GitHub supports repeatable publishing and easier updates.

Persistence:localStorage

- localStorage was used for user-specific progress (XP, settings, favorites, custom words) because authentication and database complexity were intentionally out of scope.
- This approach keeps the app fast and installation-free while preserving personalized learning state per browser.

4. AI Usage Disclosure

This section transparently documents how AI tools were used during development and what parts were manually implemented or adjusted.

4.1 Tools Used

- GitHub Copilot (GPT-5.3-Codex) for coding assistance, refactoring support, debugging guidance, and deployment command preparation.
- VS Code integrated assistant workflow for file updates, validation, and deployment troubleshooting.

4.2 What Was Generated with AI Assistance

- Initial scaffolding and updates for backend/API files (e.g., server.js and serverless endpoint structure).
- Data service abstraction (fetch logic, fallback path, status messaging).
- Deployment configuration for Vercel routing and static/API coexistence.
- README documentation updates for architecture and deployment instructions.

4.3 What Was Manually Modified / Decided by Developer

- Project direction, scope, and educational objectives (Finnish learning focus, target users, and feature priorities).
- UI/UX content decisions including wording, categories, and learner-oriented phrasing.
- Validation of deployment outcomes and debugging decisions after observing plain/no-style rendering in production.
- Final acceptance, testing, and publication decisions (what was shipped to teacher-facing URL).

4.4 Responsible AI Use Notes

AI was used as an accelerator and assistant, not as a replacement for developer judgement. Outputs were reviewed, tested, and corrected where necessary. In particular, deployment routing required manual verification and iterative adjustment to ensure CSS/JS assets were served correctly in production.

5. Reflection and Future Improvements

5.1 Reflection

The project successfully met functional and technical requirements while remaining practical for real users. A key learning point was that deployment architecture can affect runtime behavior significantly even when local tests pass. The plain-style issue observed after deployment was caused by routing behavior, not by missing CSS code. Fixing this required understanding Vercel routing precedence and separating static file handling from API/function routes.

Another reflection is that simplicity can be a strength. Using vanilla JavaScript made all application logic explicit and understandable, which is valuable in an academic setting. However, this also means maintaining larger files manually; modularization should continue as features grow.

5.2 Future Improvements

- **Expand vocabulary dataset**: Increase API data coverage beyond starter sets and include level-based packs (A1, A2, B1).
- **Add pronunciation audio**: Integrate text-to-speech or recorded clips to improve listening and speaking support.
- **Introduce spaced repetition**: Replace simple review loops with interval-based scheduling (SRS) for stronger retention.
- **Better analytics**: Track performance by category and difficulty over time with charts and weekly summaries.
- **Backend persistence**: Add optional cloud database storage for cross-device synchronization while keeping privacy controls.
- **Accessibility enhancements**: Improve keyboard navigation, ARIA labels, and contrast checks for universal usability.
- **Automated testing**: Add unit and integration tests for API endpoints and critical UI logic to reduce regression risk.

Conclusion

FinnVocab demonstrates a complete, deployable web solution for practical Finnish vocabulary learning. The application combines user-centered design, asynchronous API integration, clear architecture separation, and modern deployment practices. With transparent AI-assisted development and manual validation, the final product is suitable for academic evaluation and public demonstration.